

Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Bảy, 10 tháng 8 2024, 8:20 AM
Kết thúc lúc	Thứ Sáu, 23 tháng 8 2024, 3:54 PM
Thời gian thực hiện	13 Các ngày 7 giờ
Điểm	7,00/7,00
Điểm	10,00 trên 10,00 (100%)



Câu hỏi 1

Đúng

Đạt điểm 1,00 trên 1,00

Implement functions: **Peek, Pop, Size, Empty, Contains** to a maxHeap. If the function cannot execute, **return -1**.

```
#include <iostream>
#include <fstream>
#include <string>
#include <cstring>
#include <cmath>
#include <vector>
#include <algorithm>
using namespace std;
#define SEPARATOR "<#<ab@17943918#@>#"
template<class T>
class Heap {
protected:
    T* elements;
    int capacity;
    int count;
public:
    Heap()
    {
        this->capacity = 10;
        this->count = 0;
        this->elements = new T[capacity];
    }
    ~Heap()
    {
        delete[] elements;
    }
    void push(T item);

    bool isEmpty();
    bool contains(T item);
    T peek();
    bool pop();
    int size();

    void printHeap()
    {
        cout << "Max Heap [ ";
        for (int i = 0; i < count; i++)
            cout << elements[i] << " ";
        cout << "]\n";
    }
private:
    void ensureCapacity(int minCapacity);
    void reheapUp(int position);
    void reheapDown(int position);
};
//Your code goes here
```

For example:



Test	Result
<pre> Heap<int> maxHeap; for (int i=0;i<10;i++){ maxHeap.push(i); } cout << maxHeap.size(); </pre>	10
<pre> Heap<int> maxHeap; for (int i=0;i<10;i++){ maxHeap.push(i); } cout << maxHeap.isEmpty(); </pre>	0

Answer: (penalty regime: 0, 0, 5, ... %)

Reset answer

```

1  template<class T>
2  int Heap<T>::size(){
3      return count;
4  }
5
6  template<class T>
7  bool Heap<T>::isEmpty(){
8      return count == 0;
9  }
10
11 template<class T>
12 T Heap<T>::peek(){
13     if (isEmpty()) {
14         cout << "Heap is empty" << endl;
15         return -1; // hoặc giá trị khác tùy thuộc vào loại T
16     }
17     return elements[0];
18 }
19
20 template<class T>
21 bool Heap<T>::contains(T item){
22     for (int i = 0; i < count; i++) {
23         if (elements[i] == item) {
24             return true;
25         }
26     }
27     return false;
28 }
29
30 template<class T>
31 bool Heap<T>::pop(){
32     if (isEmpty()) {
33         cout << "Heap is empty" << endl;
34         return -1; // hoặc giá trị khác tùy thuộc vào loại T
35     }
36     T root = elements[0];
37     elements[0] = elements[count - 1];
38     count--;
39     reheapDown(0);
40     return root;
41 }

```

	Test	Expected	Got	
✓	<pre>Heap<int> maxHeap; for (int i=0;i<10;i++){ maxHeap.push(i); } cout << maxHeap.size();</pre>	10	10	✓
✓	<pre>Heap<int> maxHeap; for (int i=0;i<10;i++){ maxHeap.push(i); } cout << maxHeap.isEmpty();</pre>	0	0	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 2

Đúng

Đạt điểm 1,00 trên 1,00

Implement function push to push a new item to a maxHeap. You also have to implement ensureCapacity and reheapUp to help you achieve that.

```
template
class Heap{
protected:
    T *elements;
    int capacity;
    int count;

public:
    Heap()
    {
        this->capacity = 10;
        this->count = 0;
        this->elements = new T[capacity];
    }
    ~Heap()
    {
        delete []elements;
    }
    void push(T item);
    void printHeap()
    {
        cout << "Max Heap [ ";
        for (int i = 0; i < count; i++)
            cout << elements[i] << " ";
        cout << "]\n";
    }

private:
    void ensureCapacity(int minCapacity);
    void reheapUp(int position);
};

// Your code here
```

For example:

Test	Result
<pre>Heap<int> maxHeap; for(int i = 0; i <5;i++) maxHeap.push(i); maxHeap.printHeap();</pre>	Max Heap [4 3 1 0 2]

Answer: (penalty regime: 0, 0, 5, ... %)

Reset answer

```
1 template<class T>
2 void Heap<T>::push(T item){
3     ensureCapacity(count + 1);
4     elements[count] = item;
5     reheapUp(count);
6     count++;
7 }
8
```

```

9  template<class T>
10 void Heap<T>::ensureCapacity(int minCapacity){
11     if (minCapacity > capacity) {
12         int oldCapacity = capacity;
13         capacity = capacity + (capacity >> 1); // Tăng gấp 1.5 lần
14         if (capacity < minCapacity) {
15             capacity = minCapacity;
16         }
17         T* newElements = new T[capacity];
18         copy(elements, elements + oldCapacity, newElements);
19         delete[] elements;
20         elements = newElements;
21     }
22 }
23
24 template<class T>
25 void Heap<T>::reheapUp(int position){
26     int parent = (position - 1) / 2;
27     if (position > 0 && elements[position] > elements[parent]) {
28         swap(elements[position], elements[parent]);
29         reheapUp(parent);
30     }
31 }

```

	Test	Expected	Got	
✓	<pre> Heap<int> maxHeap; for(int i = 0; i <5;i++) maxHeap.push(i); maxHeap.printHeap(); </pre>	Max Heap [4 3 1 0 2]	Max Heap [4 3 1 0 2]	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi 3

Đúng

Đạt điểm 1,00 trên 1,00

Given an array which the elements in it are random. Now we want to build a Max heap from this array. Implement functions Reheap up and Reheap down to heapify element at index position. We will use it to build a heap in next question.

To keep things simple, this question will separate the heap array, not store it in the class heap

```
void reheapDown(int maxHeap[], int numberOfElements, int index);
void reheapUp(int maxHeap[], int numberOfElements, int index);
```

For example:

Test	Result
<pre>int arr[] = {1,2,3,4,5,6,7,8}; int size = sizeof(arr)/sizeof(arr[0]); reheapDown(arr,size,0); cout << "["; for(int i=0;i<size;i++) cout << arr[i] << " "; cout << "]"</pre>	[3 2 7 4 5 6 1 8]
<pre>int arr[] = {1,2,3,4,5,6,7,8}; int size = sizeof(arr)/sizeof(arr[0]); reheapUp(arr,size,7); cout << "["; for(int i=0;i<size;i++) cout << arr[i] << " "; cout << "]"</pre>	[8 1 3 2 5 6 7 4]

Answer: (penalty regime: 0, 0, 5, ... %)

Reset answer

```
1 void reheapDown(int maxHeap[], int numberOfElements, int index) {
2     int child = 2 * index + 1; // Left child
3     if (child >= numberOfElements) return; // If there is no left child, return
4
5     // If right child exists and is greater than left child
6     if (child + 1 < numberOfElements && maxHeap[child + 1] > maxHeap[child]) {
7         child = child + 1;
8     }
9
10    // If the largest child is greater than the current node
11    if (maxHeap[child] > maxHeap[index]) {
12        swap(maxHeap[child], maxHeap[index]);
13        reheapDown(maxHeap, numberOfElements, child);
14    }
15 }
16
17 void reheapUp(int maxHeap[], int numberOfElements, int index) {
18     if (index == 0) return; // If we are at the root, return
19
20     int parent = (index - 1) / 2; // Parent node
21     if (maxHeap[index] > maxHeap[parent]) {
22         swap(maxHeap[index], maxHeap[parent]);
23         reheapUp(maxHeap, numberOfElements, parent);
24     }
25 }
```

	Test	Expected	Got	
✓	<pre>int arr[] = {1,2,3,4,5,6,7,8}; int size = sizeof(arr)/sizeof(arr[0]); reheapDown(arr,size,0); cout << "["; for(int i=0;i<size;i++) cout << arr[i] << " "; cout << "]"</pre>	[3 2 7 4 5 6 1 8]	[3 2 7 4 5 6 1 8]	✓
✓	<pre>int arr[] = {1,2,3,4,5,6,7,8}; int size = sizeof(arr)/sizeof(arr[0]); reheapUp(arr,size,7); cout << "["; for(int i=0;i<size;i++) cout << arr[i] << " "; cout << "]"</pre>	[8 1 3 2 5 6 7 4]	[8 1 3 2 5 6 7 4]	✓
✓	<pre>int arr[] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15}; int size = sizeof(arr)/sizeof(arr[0]); reheapUp(arr,size,13); reheapUp(arr,size,12); cout << "["; for(int i=0;i<size;i++) cout << arr[i] << " "; cout << "]"</pre>	[14 2 13 4 5 1 3 8 9 10 11 12 6 7 15]	[14 2 13 4 5 1 3 8 9 10 11 12 6 7 15]	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 4

Đúng

Đạt điểm 1,00 trên 1,00

Implement method **remove** to **remove** the element with given value from a **maxHeap**, **clear** to remove all elements and bring the heap back to the initial state. You also have to implement method **getItem** to help you. Some given methods that you don't need to implement again are **push**, **printHeap**, **ensureCapacity**, **reheapUp**, **reheapDown**.

```
class Heap {
protected:
    T* elements;
    int capacity;
    int count;
public:
    Heap()
    {
        this->capacity = 10;
        this->count = 0;
        this->elements = new T[capacity];
    }
    ~Heap()
    {
        delete[] elements;
    }
    void push(T item);
    int getItem(T item);
    void remove(T item);
    void clear();
    void printHeap()
    {
        cout << "Max Heap [ ";
        for (int i = 0; i < count; i++)
            cout << elements[i] << " ";
        cout << "]\n";
    }
private:
    void ensureCapacity(int minCapacity);
    void reheapUp(int position);
    void reheapDown(int position);
};

// Your code here
```

For example:

Test	Result
<pre>Heap<int> maxHeap; int arr[] = {42,35,30,15,20,21,18,3,7,14}; for (int i = 0; i < 10; i++) maxHeap.push(arr[i]); maxHeap.remove(42); maxHeap.remove(35); maxHeap.remove(30); maxHeap.printHeap();</pre>	<pre>Max Heap [21 20 18 15 14 7 3]</pre>

Test	Result
<pre> Heap<int> maxHeap; int arr[] = {78, 67, 32, 56, 8, 23, 19, 45}; for (int i = 0; i < 8; i++) maxHeap.push(arr[i]); maxHeap.remove(78); maxHeap.printHeap(); </pre>	Max Heap [67 56 32 45 8 23 19]
<pre> Heap<int> maxHeap; int arr[] = { 13, 19, 20, 7, 15, 12, 16, 10, 8, 9, 3, 6, 18, 2, 14, 1, 17, 4, 11, 5 }; for (int i = 0; i < 20; ++i) maxHeap.push(arr[i]); maxHeap.clear(); maxHeap.printHeap(); </pre>	Max Heap []

Answer: (penalty regime: 10, 20, 30, ... %)

Reset answer

```

1  template<class T>
2  int Heap<T>::getItem(T item) {
3      if (count > 0) {
4          for (int i = 0; i < count; i++) {
5              if (elements[i] == item) {
6                  return i;
7              }
8          }
9      }
10     return -1; // Không tìm thấy phần tử hoặc heap rỗng
11 }
12
13 template<class T>
14 void Heap<T>::remove(T item) {
15     if (count == 0) return; // Nothing to remove if heap is empty
16
17     int index = getItem(item);
18     if (index == -1) return; // Item not found, nothing to remove
19
20     // Replace the item to be removed with the last item in the heap
21     elements[index] = elements[count - 1];
22     count--;
23
24     if (count > 0) { // Only reheap if there are still elements
25         // Reheap down from the replaced item's position
26         reheapDown(index);
27
28         // If the item didn't move down, we need to reheap up
29         if (index > 0 && elements[(index - 1) / 2] < elements[index]) {
30             reheapUp(index);
31         }
32     }
33 }
34
35 template<class T>
36 void Heap<T>::clear() {
37     // Delete all elements
38     delete[] elements;
39     elements = nullptr; // Set to nullptr after deleting
40     // Reallocate memory as initial state
41     this->capacity = 10;
42     this->count = 0;
43     this->elements = new T[capacity];
44 }

```

	Test	Expected	Got	
✓	<pre> Heap<int> maxHeap; int arr[] = {42,35,30,15,20,21,18,3,7,14}; for (int i = 0; i < 10; i++) maxHeap.push(arr[i]); maxHeap.remove(42); maxHeap.remove(35); maxHeap.remove(30); maxHeap.printHeap(); </pre>	Max Heap [21 20 18 15 14 7 3]	Max Heap [21 20 18 15 14 7 3]	✓
✓	<pre> Heap<int> maxHeap; int arr[] = {78, 67, 32, 56, 8, 23, 19, 45}; for (int i = 0; i < 8; i++) maxHeap.push(arr[i]); maxHeap.remove(78); maxHeap.printHeap(); </pre>	Max Heap [67 56 32 45 8 23 19]	Max Heap [67 56 32 45 8 23 19]	✓
✓	<pre> Heap<int> maxHeap; int arr[] = { 13, 19, 20, 7, 15, 12, 16, 10, 8, 9, 3, 6, 18, 2, 14, 1, 17, 4, 11, 5 }; for (int i = 0; i < 20; ++i) maxHeap.push(arr[i]); maxHeap.clear(); maxHeap.printHeap(); </pre>	Max Heap []	Max Heap []	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 5

Đúng

Đạt điểm 1,00 trên 1,00

Your task is to implement heap sort (in ascending order) on an unsorted array.

```
#define SEPARATOR "<ab@17943918#@>#"
#ifndef SORTING_H
#define SORTING_H
#include <iostream>
#include <queue>
using namespace std;
template <class T>
class Sorting {
public:
    /* Function to print an array */
    static void printArray(T *start, T *end)
    {
        long size = end - start;
        for (int i = 0; i < size - 1; i++)
            cout << start[i] << ", ";
        cout << start[size - 1];
        cout << endl;
    }

    //Helping functions go here
    static void heapSort(T* start, T* end){
        //TODO
        Sorting<T>::printArray(start,end);
    }
};
#endif /* SORTING_H */
```

For example:

Test	Result
int arr[4]={4,2,9,1}; Sorting<int>::heapSort(&arr[0],&arr[4]);	1, 2, 4, 9
int arr[4]={-1,0,2,3}; Sorting<int>::heapSort(&arr[0],&arr[4]);	-1, 0, 2, 3

Answer: (penalty regime: 0, 0, 5, ... %)

Reset answer

1

// Helping functions go here

2

static void heapify(T* start, int n, int i) {

3

int largest = i; // Initialize largest as root

4

int left = 2 * i + 1; // left = 2*i + 1

5

int right = 2 * i + 2; // right = 2*i + 2

6

7

// If left child is larger than root

8

if (left < n && start[left] > start[largest])

9

largest = left;

10

11

// If right child is larger than largest so far

12

if (right < n && start[right] > start[largest])

13

largest = right;

14

15

// If largest is not root

16

if (largest != i) {

```

17         std::swap(start[i], start[largest]);
18
19         // Recursively heapify the affected sub-tree
20         heapify(start, n, largest);
21     }
22 }
23
24 static void heapSort(T* start, T* end) {
25     int n = end - start;
26
27     // Build heap (rearrange array)
28     for (int i = n / 2 - 1; i >= 0; i--)
29         heapify(start, n, i);
30
31     // One by one extract an element from heap
32     for (int i = n - 1; i > 0; i--) {
33         // Move current root to end
34         std::swap(start[0], start[i]);
35
36         // Call max heapify on the reduced heap
37         heapify(start, i, 0);
38     }
39
40     // Print the sorted array
41     Sorting<T>::printArray(start, end);
42 }

```

	Test	Expected	Got	
✓	int arr[4]={4,2,9,1}; Sorting<int>::heapSort(&arr[0],&arr[4]);	1, 2, 4, 9	1, 2, 4, 9	✓
✓	int arr[4]={-1,0,2,3}; Sorting<int>::heapSort(&arr[0],&arr[4]);	-1, 0, 2, 3	-1, 0, 2, 3	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 6

Đúng

Đạt điểm 1,00 trên 1,00

Cho template của class PrinterQueue có 2 phương thức bắt buộc:

1. addNewRequest(int priority, string fileName)

Phương thức đầu tiên sẽ thêm 1 file vào danh sách hàng đợi của máy in (bao gồm độ ưu tiên và tên file). Test case sẽ có tối đa 100 file cùng lúc trong hàng đợi

2. print()

Phương thức thứ hai sẽ in tên file kèm xuống dòng và xóa nó ra khỏi hàng đợi. Nếu không có file nào trong hàng đợi, phương thức sẽ in ra "No file to print" kèm xuống dòng.

PrinterQueue tuân theo các quy tắc sau:

- fileName có độ ưu tiên cao nhất sẽ được in trước.
- Các fileName có cùng độ ưu tiên sẽ in theo thứ tự FIFO (First In First Out) order.

Nhiệm vụ của bạn là hiện thực class PrinterQueue thỏa mãn các yêu cầu dữ liệu trên

Lưu ý: Bạn có thể thay đổi mọi thứ, thêm thư viện cần thiết ngoại trừ thay đổi tên class, prototype của 2 public method bắt buộc.

Giải thích testcase 1: File goodbye.pdf có độ ưu tiên là 2 và được thêm vào sớm hơn file goodnight.pdf (độ ưu tiên = 2) nên sẽ được in trước, sau đó đến file goodnight.pdf và cuối cùng là hello.pdf có độ ưu tiên thấp nhất (1)

For example:

Test	Result
<pre>PrinterQueue* myPrinterQueue = new PrinterQueue(); myPrinterQueue->addNewRequest(1, "hello.pdf"); myPrinterQueue->addNewRequest(2, "goodbye.pdf"); myPrinterQueue->addNewRequest(2, "goodnight.pdf"); myPrinterQueue->print(); myPrinterQueue->print(); myPrinterQueue->print();</pre>	<pre>goodbye.pdf goodnight.pdf hello.pdf</pre>
<pre>PrinterQueue* myPrinterQueue = new PrinterQueue(); myPrinterQueue->addNewRequest(1, "hello.pdf"); myPrinterQueue->print(); myPrinterQueue->print(); myPrinterQueue->print();</pre>	<pre>hello.pdf No file to print No file to print</pre>

Answer: (penalty regime: 0, 0, 0, 100 %)

Reset answer

```
1 #include <iostream>
2 #include <string>
3 #include <map>
4 #include <queue>
5
6 class PrinterQueue {
7 private:
8     // Map to store files with priority and their order of insertion
9     std::map<int, std::queue<std::string>> queue;
10
11 public:
12     // Add a new print request
13     void addNewRequest(int priority, std::string fileName) {
14         queue[priority].push(fileName);
15     }
```

```
16
17 // Print the file with the highest priority
18 void print() {
19     if (!queue.empty()) {
20         // Get the iterator to the last element (highest priority)
21         auto it = --queue.end();
22
23         // Print the file name at the front of the queue
24         std::cout << it->second.front() << std::endl;
25
26         // Remove the printed file from the queue
27         it->second.pop();
28
29         // If the queue for that priority is now empty, remove the priority from the map
30         if (it->second.empty()) {
31             queue.erase(it);
32         }
33     } else {
34         std::cout << "No file to print" << std::endl;
35     }
36 }
37 };
```

	Test	Expected	Got	
✓	PrinterQueue* myPrinterQueue = new PrinterQueue(); myPrinterQueue->addNewRequest(1, "hello.pdf"); myPrinterQueue->addNewRequest(2, "goodbye.pdf"); myPrinterQueue->addNewRequest(2, "goodnight.pdf"); myPrinterQueue->print(); myPrinterQueue->print(); myPrinterQueue->print();	goodbye.pdf goodnight.pdf hello.pdf	goodbye.pdf goodnight.pdf hello.pdf	✓
✓	PrinterQueue* myPrinterQueue = new PrinterQueue(); myPrinterQueue->addNewRequest(1, "hello.pdf"); myPrinterQueue->print(); myPrinterQueue->print(); myPrinterQueue->print();	hello.pdf No file to print No file to print	hello.pdf No file to print No file to print	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 7

Đúng

Đạt điểm 1,00 trên 1,00

Given an array of non-negative integers. Each time, we can take the smallest integer out of the array, multiply it by 2, and push it back to the array.

Request: Implement function:

```
int leastAfter(vector<int>& nums, int k);
```

Where `nums` is the given array (the length of the array is between 1 and 100000). This function returns the smallest integer in the array after performing the operation `k` times (`k` is between 1 and 100000).

Example:

Given `nums = [2, 3, 5, 7]`.

In the 1st operation, we take 2 out and push back 4. The array is now `nums = [3, 4, 5, 7]`.

In the 2nd operation, we take 3 out and push back 6. The array is now `nums = [4, 5, 6, 7]`.

In the 3rd operation, we take 4 out and push back 8. The array is now `nums = [5, 6, 7, 8]`.

With `k = 3`, the result would be 5.

Note:

In this exercise, the libraries `iostream`, `string`, `cstring`, `climits`, `utility`, `vector`, `list`, `stack`, `queue`, `map`, `unordered_map`, `set`, `unordered_set`, `functional`, `algorithm` has been included and `namespace std` are used. You can write helper functions and classes. Importing other libraries is allowed, but not encouraged, and may result in unexpected errors.

For example:

Test	Result
<pre>vector<int> nums {2, 3, 5, 7}; int k = 3; cout << leastAfter(nums, k);</pre>	5

Answer: (penalty regime: 0, 0, 0, 5, 10, ... %)

Reset answer

```
1 #include <queue> // For priority_queue
2
3 int leastAfter(std::vector<int>& nums, int k) {
4     // Create a min-heap using a priority queue
5     std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap(nums.begin(), nums.end());
6
7     // Perform the operation k times
8     for (int i = 0; i < k; ++i) {
9         int smallest = minHeap.top(); // Get the smallest element
10        minHeap.pop(); // Remove the smallest element
11        minHeap.push(smallest * 2); // Push the doubled value back into the heap
12    }
13
14    // The smallest element after k operations
15    return minHeap.top();
16 }
```

	Test	Expected	Got	
✓	<pre>vector<int> nums {2, 3, 5, 7}; int k = 3; cout << leastAfter(nums, k);</pre>	5	5	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

